

Task 4: Order Management System

Aim:

To design and implement an Order Management System using SQL by creating Customers, Orders, and Products tables, and to retrieve meaningful insights using JOINS, Subqueries, and ORDER BY.

Procedure:

1. Create the database tables:
 - Customers
 - Products
 - Orders
2. Insert sample data into all tables.
3. Use JOIN queries to display customer order history.
4. Use subqueries to:
 - Find the highest value order.
 - Identify the most active customer.
5. Sort results using ORDER BY.
6. Display outputs.

Algorithm:

1. Start
2. Create tables: Customers, Products, Orders
3. Insert sample records
4. Perform JOIN operation:
 - Join Customers → Orders → Products
5. Use subquery to:
 - Find maximum order value
 - Find customer with highest number of orders
6. Display results in sorted order
7. Stop

Program:

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(50),  
    price DECIMAL(10,2)  
);
```

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    product_id INT,  
    quantity INT,  
    order_date DATE,  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),  
    FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);
```

INSERT INTO Customers VALUES

```
(1, 'Rahul', 'Chennai'),  
(2, 'Anita', 'Bangalore'),  
(3, 'Kiran', 'Hyderabad');
```

INSERT INTO Products VALUES

```
(101, 'Laptop', 50000),  
(102, 'Mobile', 20000),  
(103, 'Headphones', 2000);
```

INSERT INTO Orders VALUES

```
(1, 1, 101, 1, '2024-01-10'),  
(2, 1, 103, 2, '2024-02-15'),  
(3, 2, 102, 1, '2024-03-05'),  
(4, 3, 103, 5, '2024-03-20'),  
(5, 2, 101, 1, '2024-04-01');
```

```
SELECT
    c.name,
    p.product_name,
    o.quantity,
    (p.price * o.quantity) AS total_amount,
    o.order_date
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Products p ON o.product_id = p.product_id
ORDER BY o.order_date;
```

```
SELECT * FROM (
    SELECT
        o.order_id,
        c.name,
        (p.price * o.quantity) AS total_value
    FROM Orders o
    JOIN Customers c ON o.customer_id = c.customer_id
    JOIN Products p ON o.product_id = p.product_id
) AS order_values
WHERE total_value = (
    SELECT MAX(p.price * o.quantity)
    FROM Orders o
    JOIN Products p ON o.product_id = p.product_id
);
```

```
SELECT name FROM Customers
WHERE customer_id = (
    SELECT customer_id
    FROM Orders
    GROUP BY customer_id
    ORDER BY COUNT(order_id) DESC
    LIMIT 1
);
```

Output:

1. Customer Order History

name	product_name	quantity	total_amount	order_date
Rahul	Laptop	1	50000.00	2024-01-10
Rahul	Headphones	2	4000.00	2024-02-15
Anita	Mobile	1	20000.00	2024-03-05
Kiran	Headphones	5	10000.00	2024-03-20
Anita	Laptop	1	50000.00	2024-04-01

5 rows in set (0.06 sec)

2. Highest Value Order

order_id	name	total_value
1	Rahul	50000.00
5	Anita	50000.00

2 rows in set (0.01 sec)

3. Most Active Customer

name
Rahul

1 row in set (0.01 sec)

Result:

The Order Management System was successfully implemented using SQL.

- JOIN operations helped in combining data from multiple tables.
- Subqueries were used to identify key insights like highest order value and most active customer.
- The system demonstrates real-world usage in e-commerce and retail applications.

